PUBLIC

Code Assessment

of the Spark ALM Controller Smart Contracts

February 17, 2026

Produced for

Spark

by

CHAINSECURITY

Contents

1 Executive Summary 3
2 Assessment Overview 5
3 Limitations and use of report 9
4 Terminology 10
5 Open Findings 11
6 Resolved Findings 12
7 Informational 14

1 Executive Summary

Dear all,

Thank you for trusting us to help SparkDAO with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Spark ALM Controller according to Scope to support you in forming an opinion on their security risks.

SparkDAO implements Spark ALM Controller, a set of on-chain components of the Spark Liquidity Layer designed to manage and control the flow of liquidity on Ethereum mainnet and L2s, and between them, by leveraging DSS Allocator.

This assessment is a **differential review** performed on the source code files inside the Spark ALM Controller repository, covering only the changes introduced between version v1.9.0 (3dbc7cb01739e91dad61a75cda8d7c84b4474e0b) and version v1.10.

The most critical subjects covered in our audit are functional correctness, access control, and the integration with 3rd-party protocols. The general subjects covered are gas efficiency, documentation and composability. Security regarding all the aforementioned subjects is high.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

This assessment is a **differential review** performed on the source code files inside the Spark ALM Controller repository, covering only the changes introduced between version v1.9.0 (3dbc7cb01739e91dad61a75cda8d7c84b4474e0b) and version v1.10-beta.0.

Important: This review is conducted under the assumption that the codebase at v1.9.0 is correct and secure. Only the modifications, additions, and deletions introduced in the commits between these two versions are considered in scope. Findings from previous audits and the security properties established in v1.9.0 are assumed to hold.

The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	30 Jan 2026	5379267e82cd55de507bb253b93c7831d9d99595	v1.10-beta.0
2	16 Feb 2026	984ec546fb2c98ed729ae91d2d73e97dedbf111f	v1.10

For the solidity smart contracts, the compiler version `0.8.25` was chosen with the evm version set to `cancun`.

The following new source files were added and are fully in scope:

```
src/
    WEETHModule.sol
    libraries/
        LayerZeroLib.sol
        WEETHLib.sol
```

Only the changes introduced since v1.9.0 are considered in scope for the following files:

```
src/
    MainnetController.sol
    ForeignController.sol
    OTCBuffer.sol
    libraries/
        ApproveLib.sol
        CurveLib.sol
        ERC4626Lib.sol
        PSMLib.sol
        UniswapV4Lib.sol
```

2.1.1 Excluded from scope

Any other files are out of scope.

The following are explicitly out of scope:

- The entire codebase as reviewed in previous versions (v1.0.0 through v1.9.0) - these are assumed correct and secure
- All 3rd-party protocols that the ALMProxy integrates with are out of scope. They are assumed to work honestly and correctly as documented.
- Dependencies and libraries e.g. OpenZeppelin.

2.2 System Overview

This system overview describes the changes introduced between version v1.9.0 and version v1.10-beta.0 of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

SparkDAO offers Spark ALM Controller, a set of on-chain components of the Spark Liquidity Layer designed to manage and control the flow of liquidity on Ethereum mainnet and L2s, and between them, by leveraging DSS Allocator.

On each chain, the following contracts are deployed:

1. ALMProxy: Entity that holds funds and interacts with external contracts (e.g. DssAllocator, PSM). Thus, it holds the required privileges to interact with other contracts.
2. Controllers: Dictate which operations an ALMProxy shall perform. Note that multiple controllers could point to the same ALMProxy.
3. RateLimits: Computes limits for liquidity flows.

All of the contracts inherit the standard AccessControl and grant the *DEFAULT_ADMIN_ROLE* role to an `admin` which can configure other roles.

The following changes were introduced in v1.10-beta.0. For full reviews of previous releases, please refer to Spark ALM Controller Audits.

2.2.1 New Integrations

2.2.1.1 Etherfi Integration

Introduced in this version; only in **mainnet** controller.

It enables the ALMProxy to deposit into and withdraw from EtherFi's liquidity pool. The integration consists of two components:

WEETHLib: A library providing the following functions:

1. `deposit()`: Converts WETH to weETH. The function unwraps WETH, deposits ETH into EtherFi's liquidity pool to receive eETH shares, then wraps the eETH into weETH. A `minSharesOut` parameter provides slippage protection.
2. `requestWithdraw()`: Initiates a withdrawal from weETH by unwrapping weETH to eETH, then requesting a withdrawal from the EtherFi liquidity pool. An NFT receipt is minted to the `WEETHModule` contract representing the withdrawal request. Since Version 2, a `minEETHShares` parameter is defined for weETH unwrapping.
3. `claimWithdrawal()`: Claims a finalized withdrawal request, receiving ETH which is then wrapped to WETH and transferred to the ALMProxy.

WEETHModule: An upgradeable UUPS proxy contract that:

- Implements `onERC721Received()` and acts as the recipient for withdrawal request NFTs.

- Provides `claimWithdrawal()` function that claims ETH from finalized requests, wraps to WETH, and transfers to ALMProxy
- Restricted so only the ALMProxy can call `claimWithdrawal()`
- Only `DEFAULT_ADMIN_ROLE` can upgrade the implementation

Rate limits apply to all three operations: deposit, request withdrawal, and claim withdrawal.

MainnetController Functions:

- `depositToWeETH(uint256 amount, uint256 minSharesOut)`: Deposits WETH to weETH with slippage protection.
- `requestWithdrawFromWeETH(address weETHModule, uint256 weETHShares)`: Initiates withdrawal request.
- `claimWithdrawalFromWeETH(address weETHModule, uint256 requestId)`: Claims finalized withdrawal.

2.2.2 Fixes and Refactoring

2.2.2.1 LayerZeroLib

Refactored in this version; extracted from both `MainnetController` and `ForeignController`.

The LayerZero integration logic has been extracted into a shared library `LayerZeroLib.sol`. The library provides `transferTokenLayerZero()`: Handles the complete LayerZero OFT bridging flow including rate limiting, approval handling for OFTs that require it, quote retrieval, and execution of the send operation.

2.2.2.2 Upgradeable OTCBuffer

`OTCBuffer`, which acts as a buffer to receive the finalized OTC swap, has been converted from an immutable contract to a UUPS upgradeable proxy, which can be upgraded by its admin.

2.2.2.3 ERC4626 Safety Parameters

All ERC4626-related functions in both controllers now include slippage protection parameters:

- `depositERC4626`: Added `minSharesOut` - reverts if shares received is less than specified
- `withdrawERC4626`: Added `maxSharesIn` - reverts if shares burned exceeds specified
- `redeemERC4626`: Added `minAssetsOut` - reverts if assets received is less than specified

These parameters prevent sandwich attacks and unfavorable exchange rate scenarios.

In addition, `wrapAllProxyETH()` has been added to `MainnetController` that wraps all ETH held by the ALMProxy to WETH.

2.2.3 Changelog

In **Version 2**:

1. A `minEETHShares` parameter is defined for weETH unwrapping in `WEETHLib.requestWithdraw()`.
2. `WEETHLib.claimWithdrawal()` now only requires a rate limit existing for `requestWithdraw()`.

2.3 Trust Model Additions

The following trust assumptions are added for the new components:

- **WEETHModule:** Its *admin* (`DEFAULT_ADMIN_ROLE`) is fully trusted and can upgrade the implementation. In the worst case it can steal the funds in the pending withdraw request NFTs it owns.
- **Upgradeable OTCBuffer:** The *admin* is fully trusted and can upgrade the implementation. In the worst case it can steal funds held by the buffer.

weETH (EtherFi) Integration:

- EtherFi protocol is assumed to work as documented.
- EtherFi governance is trusted to not misbehave. They can DoS the system with pausing or other admin privileges. In the worst case, they can upgrade the EtherFi contracts to steal the funds.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the Resolved Findings section. The findings are split into these different categories:

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the Open Findings section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0
Informational Findings	2

- Inconsistent LayerZero OFT Quote Caller Code Corrected
- increasePosition Uses Wrong Settlement Action Code Corrected

6.1 Inconsistent LayerZero OFT Quote Caller

Informational Version 1 Code Corrected

CS-SPKALMEXTRA-004

In `transferTokenLayerZero()` the following inconsistency exists:

1. It invokes `proxy.doCall()` to perform a `oft.quoteOFT` call from the ALMProxy to get the `OFTReceipt`.
2. However, it invokes `oft.quoteSend()` directly in the name of the Controller to get the `MessagingFee`.

There is an inconsistency in callers. In case `quoteSend` is `msg.sender` dependent, the quoted `MessagingFee` might be incorrect for the actual transfer executed by the ALMProxy.

Code corrected:

Code has been adapted and now invokes `proxy.doCall()` to perform the `oft.quoteSend` call from the ALMProxy.

6.2 `increasePosition` Uses Wrong Settlement Action

Informational Version 1 Code Corrected

CS-SPKALMEXTRA-003

In `UniswapV4Lib`, `increasePosition()` uses `Actions.SETTLE_PAIR` for settlement:

```
// UniswapV4Lib.sol:454-457
bytes memory actions = abi.encodePacked(
    uint8(Actions.INCREASE_LIQUIDITY),
```

```
    uint8(Actions.SETTLE_PAIR)
);
```

`SETTLE_PAIR` calls `_getFullDebt()` which reverts on positive deltas:

```
// DeltaResolver.sol:60-66
function _getFullDebt(Currency currency) internal view returns (uint256 amount) {
    int256 _amount = poolManager.currencyDelta(address(this), currency);
    if (_amount > 0) revert DeltaNotNegative(currency);
    amount = uint256(-_amount);
}
```

When increasing liquidity on an existing position, `_modifyLiquidity` returns both the liquidity delta and fees accrued:

```
// PositionManager.sol:297-300
(BalanceDelta liquidityDelta, BalanceDelta feesAccrued) =
    _modifyLiquidity(info, poolKey, liquidity.toInt256(), bytes32(tokenId), hookData);
(liquidityDelta - feesAccrued).validateMaxIn(amount0Max, amount1Max);
```

The net currency delta is `liquidityDelta + feesAccrued`. In rare cases where `feesAccrued` is large enough to offset the negative `liquidityDelta`, the net delta becomes positive, causing `_getFullDebt()` to revert with `DeltaNotNegative`.

Code corrected:

Code has been adapted to use `Actions.CLOSE_CURRENCY` for settlement.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 Dust eETH Shares When Wrapping Into weETH

Informational **Version 1** **Acknowledged**

CS-SPKALMEXTRA-001

In `WEETHLib`, when wrapping eETH into weETH, the following steps are performed:

1. Convert eETH shares to an eETH amount using `liquidityPool.amountForShare()`, which performs the following share to underlying computation that rounds down.

```
eETHAmount = (_share * getTotalPooledEther()) / totalShares;
```

1. Call `WEETH.wrap()` with eETHAmount from the previous step, which computes the shares (rounding down) to be transferred as follows:

```
shares = (_amount * eETH.totalShares()) / totalPooledEther;
```

Consequently, not all the eETH shares may be wrapped due to rounding errors in the conversion. Consider the following examples:

```
# Example 1
Pool State:
    totalShares:        1000000000000000007
    totalPooledEther:   1000000000000000003
    ExchangeRate:       ~1.0
User State:
    userShares:         1000000000000000000
    userBalanceEther:   99999999999999999
    sharesToTransfer:   99999999999999999
    dust:               1

# Example 2
Pool State:
    totalShares:      816116506552259432609369 8
    totalPooledEther: 446965807297444879060172 8
    ExchangeRate:     0.547674
User State:
    userShares:       765401968638904196136 96
    userBalanceEther: 419190772485416859646 72
    sharesToTransfer: 765401968638904196136 94
    dust:             2
```

7.2 `setOTCRechargeRate` May Flip OTC Swap Status

Informational **Version 1** **Acknowledged**

CS-SPKALMEXTRA-002

`setOTCRechargeRate()` changes the recharge rate without checking if there is an ongoing OTC swap. When called during an active swap, the new rate is applied retroactively to calculate the swap's readiness status, which can cause `isOtcSwapReady()` to flip immediately.

This occurs because `getOtcClaimWithRecharge()` uses the current `rechargeRate18` for the entire elapsed time since `sentTimestamp`:

```
otc.claimed18 + (block.timestamp - otc.sentTimestamp) * otc.rechargeRate18;
```